

UNIVERSIDAD PERUANA LOS ANDES

FACULTAD DE INGENIERÍA

**ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS Y
COMPUTACIÓN**



TEMA:

EVIDENCIA SEMANA 01 ACTIVIDAD 02 - 2026

CURSO:

ALGORITMOS Y ESTRUCTURA DE DATOS

CICLO:

V

ALUMNO:

TAIPE SULLA, FERNANDO

HUANCAYO – PERU

2026

Índice del documento

Sección	Contenido
I	Introducción
II	Objetivos (general y específicos)
III	Enunciado de la actividad y análisis del problema
IV	Fundamento teórico: los cuatro pilares de la POO
V	Diseño de la solución (diagrama de clases y algoritmo)
VI	Explicación del código paso a paso
VII	Evidencias de ejecución
VIII	Resultados y verificación de los cálculos
IX	Conclusiones
X	Recomendaciones y mejoras futuras
XI	Anexo A: código fuente completo

Sugerencia: si publicas el PDF en tu portafolio web, usa esta misma numeración romana como anclas o títulos de las secciones de la página.

I. Introducción

El presente trabajo documenta el desarrollo de una aplicación de consola escrita en **Java** sobre el entorno **NetBeans IDE**, cuyo propósito es registrar las notas de las prácticas de un alumno y calcular su promedio final aplicando una regla muy frecuente en el ámbito académico: **se elimina la práctica con la nota más baja** antes de promediar.

El punto de partida fue un proyecto previo de manejo de **arreglos** (creación, ingreso, listado, eliminación, inserción, suma, menor, mayor y promedio). Ese proyecto se reorganizó y se amplió para que, además de resolver la actividad solicitada, evidencie de manera explícita los cuatro pilares de la Programación Orientada a Objetos: **abstracción, encapsulamiento, herencia y polimorfismo**.

La solución no se limita a calcular un número. Incorpora una **validación estricta del ingreso de datos** (no se acepta texto donde se espera un número, ni notas fuera del rango 0-20, ni campos vacíos, ni operaciones sobre un arreglo aún no creado), de modo que el programa nunca se interrumpe por un error del usuario. Todas las evidencias mostradas en la sección VII corresponden a ejecuciones reales del programa.

II. Objetivos

2.1 Objetivo general

Desarrollar una solución en Java que permita ingresar, almacenar y procesar las notas de las prácticas de un alumno, obteniendo el promedio redondeado luego de eliminar la nota más baja, aplicando los principios de la Programación Orientada a Objetos.

2.2 Objetivos específicos

- Diseñar una jerarquía de clases que represente un arreglo de enteros y especializarla hasta obtener un arreglo de notas con reglas propias.
- Almacenar las notas de todas las prácticas en un arreglo de tamaño definido por el usuario.

- Determinar la nota más baja, excluirla del cálculo y obtener el promedio de las prácticas restantes.
- Mostrar el promedio redondeado al entero más cercano e indicar la condición del alumno.
- Validar todo ingreso de datos: tipo de dato, rango permitido, campos vacíos y estado del arreglo.
- Evidenciar en el código la abstracción, el encapsulamiento, la herencia y el polimorfismo (estático y dinámico).
- Documentar el proceso con evidencias de ejecución y verificación matemática de los resultados para su publicación en un portafolio web.

III. Enunciado de la actividad y análisis del problema

3.1 Enunciado

Actividad: Desarrollar una solución que permita ingresar y almacenar las notas de todas las prácticas de un alumno, determinar el promedio de las prácticas sabiendo que se elimina la práctica con la nota más baja, mostrar el promedio redondeado y validar el ingreso de datos.

3.2 Descomposición en requerimientos

N	Requerimiento del enunciado	Como se resuelve en el programa
R1	Ingresar las notas de todas las prácticas	Opción 3 del menú. El método ingresarNotas() repite la lectura hasta llenar el arreglo.
R2	Almacenar las notas	Arreglo int[] numerito encapsulado en ArregloBase, con un tope que indica cuantas posiciones están ocupadas.
R3	Determinar el promedio eliminando la nota más baja	$\text{promedioSinMenor}() = (\text{suma total} - \text{nota menor}) / (\text{cantidad de prácticas} - 1)$.
R4	Mostrar el promedio redondeado	promedioRedondeado() aplica Math.round() y devuelve un entero.
R5	Validar el ingreso de datos	leerEntero(), leerEnteroEnRango(), leerTexto() y los métodos verifica...() de ArregloBase.

3.3 Reglas de validación definidas

Dato	Regla aplicada	Mensaje / acción
Opción del menú	Entero entre 0 y 7	Vuelve a pedir la alternativa
Nombres, apellidos, código, curso	Texto no vacío	"El dato no puede quedar vacío."
Cantidad de prácticas	Entero entre 2 y 20	Se exige un mínimo de 2 porque una práctica se elimina
Nota de práctica	Entero entre 0 y 20	"Fuera de rango. El valor debe estar entre 0 y 20."
Texto donde se espera un número	Se captura NumberFormatException	"Dato no válido. Ingrese un número entero." (el programa no se cae)
Operar sin alumno registrado	Se verifica que el objeto exista	"Primero debe registrar los datos del alumno (opción 1)."
Operar sin arreglo creado o sin datos	verificaCreado() / verificaConDatos()	Avisa al usuario y no ejecuta la operación
Promedio con menos de 2 notas	Se comprueba tope ≥ 2	No se puede eliminar la más baja; se informa al usuario

3.4 Fórmula utilizada

Si n es la cantidad de prácticas registradas, x_i la nota de la práctica i , y min la nota más baja, el promedio pedido es:

$$\text{Promedio} = (x_1 + x_2 + \dots + x_n) - \min / (n - 1)$$

$$\text{Promedio final} = \text{redondeo}(\text{Promedio})$$

El redondeo se realiza con `Math.round()`, que aproxima al entero más cercano y eleva los valores terminados en 0.5 (por ejemplo 14.50 se convierte en 15). Por eso se exige un mínimo de 2 prácticas: si solo hubiera una y se eliminara la más baja, no quedarían notas para promediar (división entre cero).

IV. Fundamento teórico: los cuatro pilares de la POO

La Programación Orientada a Objetos organiza un programa en **clases** (plantillas) y **objetos** (instancias de esas plantillas). A continuación se define cada pilar y se indica exactamente donde aparece en este proyecto.

4.1 Abstracción

Consiste en representar únicamente las características esenciales de un elemento del mundo real, ocultando los detalles internos. En Java se logra con **clases abstractas** e **interfaces**, que definen *que* se debe hacer sin decir *como*.

- **ArregloBase** es una clase abstracta: describe la idea de "arreglo de enteros con tope" y declara el método abstracto `mostrarInformacion()`. No puede instanciarse.
- **Calculable** es una interfaz: exige `sumar()`, `hallarMenor()`, `hallarMayor()`, `hallarPromedio()` y `sumarImpares()`, sin indicar la implementación.
- **Persona** es abstracta: un "alguien" generico del sistema académico que obliga a definir `getRol()` y `mostrarDatos()`.

4.2 Encapsulamiento

Consiste en proteger los datos de un objeto declarandolos **private** y permitir el acceso solo mediante métodos publicos (getters y setters) que validan la información antes de modificar el estado interno.

- Los atributos `numerito` y `tope` de `ArregloBase` son `private`; nadie los modifica directamente.
- `setTope()` rechaza un valor negativo o mayor que la capacidad y lanza `IllegalArgumentException`.
- `Persona` y `Alumno` validan en sus setters que nombres, apellidos, código y curso no esten vacios.

4.3 Herencia

Permite que una clase hija reutilice los atributos y métodos de una clase padre y agregue o modifique comportamiento. En este proyecto existen dos jerarquías, una de ellas **multinivel**.

Jerarquía	Cadena de herencia	Que aporta cada nivel
Arreglos	<code>ArregloBase</code> -> <code>Arreglos</code> -> <code>ArregloEstadistico</code> -> <code>ArregloNotas</code>	Estructura y lectura/escritura; edición de elementos; cálculos estadísticos; reglas de notas y promedio sin la más baja.
Personas	<code>Persona</code> -> <code>Alumno</code> / <code>Docente</code>	Datos comunes de una persona; el alumno agrega código, curso y sus prácticas; el docente agrega especialidad.

4.4 Polimorfismo

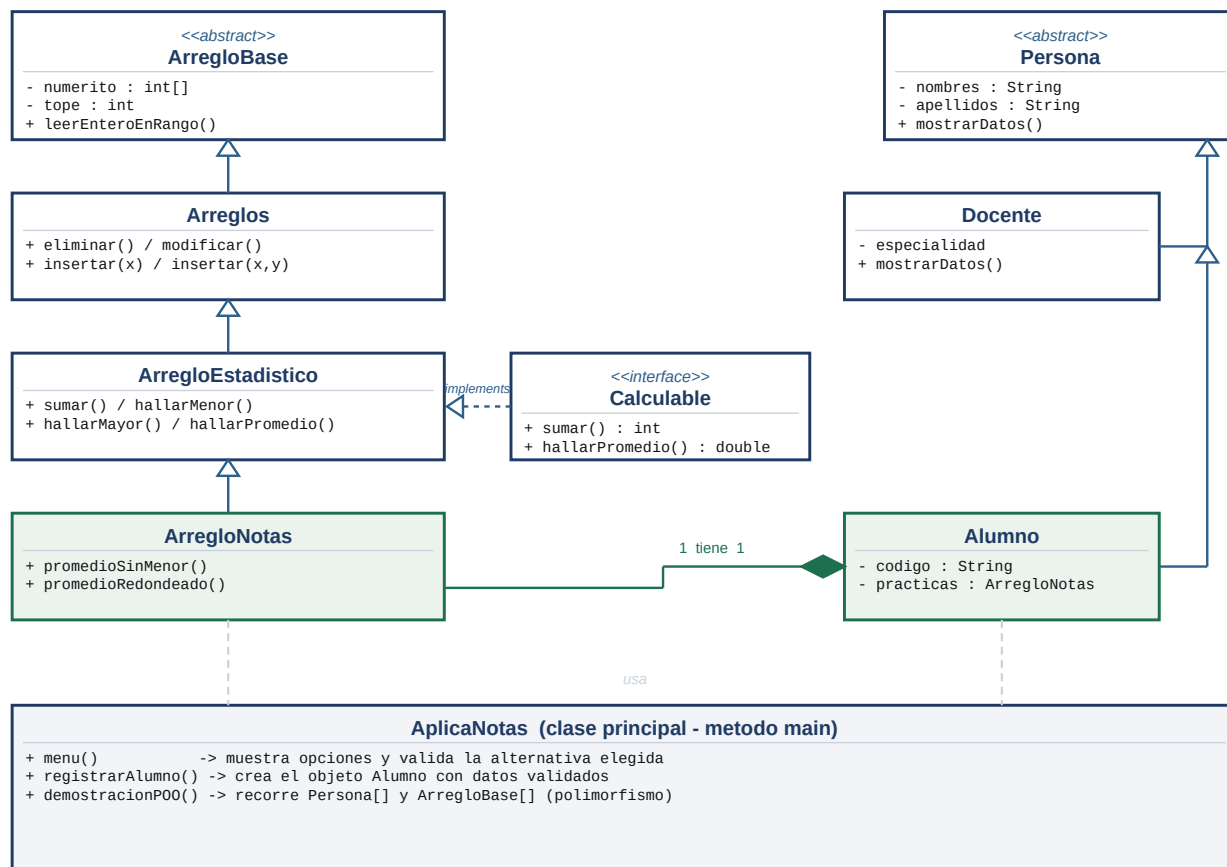
Es la capacidad de que un mismo mensaje produzca comportamientos distintos según el objeto que lo recibe. Se aplica de dos formas:

Tipo	Mecanismo	Ejemplo en el proyecto
Estático (sobrecarga)	Mismo nombre de método, distinta lista de parámetros	<code>promedioRedondeado()</code> devuelve un entero; <code>promedioRedondeado(2)</code> devuelve un double con 2 decimales. También <code>insertar(x)</code> e <code>insertar(x, y)</code> .
Dinámico (sobrescritura)	La clase hija redefine un método del padre con <code>@Override</code>	<code>getNumero()</code> en <code>ArregloNotas</code> pide una nota validada 0-20; <code>escritura()</code> imprime una tabla Práctica/Nota; <code>mostrarDatos()</code> se comporta distinto en <code>Alumno</code> y en <code>Docente</code> .

Tipo	Mecanismo	Ejemplo en el proyecto
Por referencia	Una variable del tipo padre apunta a objetos hijos	Persona[] personas = {alumno, docente}; ArregloBase[] estructuras = {...}. El mismo bucle produce salidas diferentes.

V. Diseño de la solución

5.1 Diagrama de clases



Las flechas con punta triangular hueca indican herencia (extends); la línea punteada indica implementación de la interfaz (implements); el rombo verde indica composición: un Alumno contiene su propio objeto ArregloNotas.

5.2 Responsabilidad de cada clase

Clase	Tipo	Responsabilidad principal
ArregloBase	Abstracta	Guarda el arreglo int[] y el tope de forma encapsulada. Provee la lectura validada de datos (leerEntero, leerEnteroEnRango, leerTexto) y las verificaciones de estado.
Arreglos	Concreta	Operaciones que modifican el arreglo: eliminar, modificar, insertar (sobrecargado) y mover elementos.
Calculable	Interfaz	Contrato de cálculo: sumar, hallarMenor, hallarMayor, hallarPromedio, sumarImpares.
ArregloEstadistico	Concreta	Implementa Calculable: recorre el arreglo para obtener suma, menor, mayor y promedio simple.
ArregloNotas	Concreta	Especializa el arreglo como notas 0-20. Calcula el promedio eliminando la nota más baja y lo redondea. Es el corazón de la actividad.
Persona	Abstracta	Nombres y apellidos validados; obliga a definir el rol y la forma de mostrarse.
Alumno	Concreta	Código, curso y el objeto ArregloNotas con sus prácticas; imprime la ficha completa.
Docente	Concreta	Especialidad; sirve para demostrar el polimorfismo con Persona[].
AplicaNotas	Principal	Contiene el método main y el menú de opciones; coordina todos los objetos.

5.3 Algoritmo del cálculo solicitado

Paso	Acción
1	Verificar que exista un alumno registrado y que el arreglo tenga notas.
2	Comprobar que haya al menos 2 prácticas (si no, no se puede eliminar ninguna).
3	Recorrer el arreglo acumulando la suma total de las notas.
4	Recorrer el arreglo buscando la nota menor y su posición.
5	Restar la nota menor a la suma total.
6	Dividir ese resultado entre (cantidad de prácticas - 1) usando división real.
7	Redondear el promedio con Math.round() al entero más cercano.
8	Mostrar el detalle del cálculo, el promedio redondeado y la condición del alumno.

VI. Explicación del código paso a paso

A continuación se presenta el código por partes, en el mismo orden en que conviene leerlo. El código fuente completo, sin recortes, se encuentra en el Anexo A.

6.1 ArregloBase: abstracción y encapsulamiento

Es la base de toda la jerarquía. Al declararse **abstract** no se puede escribir `new ArregloBase()`: solo sirve como plantilla para sus hijas.

```
// Atributos encapsulados
private int[] numerito;
private int tope;

// Constructor por defecto
public ArregloBase() {
    this.numerito = null;
    this.tope = 0;
}

// Constructor con parametro
public ArregloBase(int valor) {
    if (valor < 1) {
        throw new IllegalArgumentException("El tamaño debe ser mayor que 0.");
    }
    this.numerito = new int[valor];
    this.tope = 0;
}
```

Paso 1. Los atributos son **private**: el arreglo de enteros y el tope, que indica cuantas posiciones están realmente ocupadas. El constructor con parámetro rechaza tamaños menores que 1 y deja el tope en cero, porque el arreglo se crea vacío.

```
public int getTope() {
    return tope;
}

public void setTope(int tope) {
    int capacidad = (numerito == null) ? 0 : numerito.length;
    if (tope < 0 || tope > capacidad) {
        throw new IllegalArgumentException("Tope fuera de rango: " + tope);
    }
    this.tope = tope;
}

public int getCapacidad() {
    return (numerito == null) ? 0 : numerito.length;
}

// Metodo para crear el arreglo
```

Paso 2. El setter no asigna a ciegas: calcula la capacidad real del arreglo y lanza una excepción si el valor recibido esta fuera de rango. Este es el encapsulamiento en acción, el objeto se protege a si mismo de quedar en un estado inválido.

Validación del ingreso de datos

```
/**
 * Lee una línea completa y la convierte a entero. Si el usuario escribe
 * letras, símbolos o deja vacío, vuelve a preguntar (no se cae el programa).
 */
public static int leerEntero(String mensaje) {
    while (true) {
        System.out.print(mensaje);
        String linea = TECLADO.nextLine().trim();
        try {
            return Integer.parseInt(linea);
        } catch (NumberFormatException e) {
            System.out.println(">> Dato no valido: \"" + linea + "\". Ingrese un numero entero.");
        }
    }
}
```

```

/** Lee un entero y ademas valida que este dentro del rango [min, max]. */
public static int leerEnteroEnRango(String mensaje, int min, int max) {
    while (true) {
        int valor = leerEntero(mensaje);
        if (valor >= min && valor <= max) {
            return valor;
        }
        System.out.println(">> Fuera de rango. El valor debe estar entre " + min + " y " + max + ".");
    }
}

/** Lee un texto no vacio. */
public static String leerTexto(String mensaje) {
    while (true) {
        System.out.print(mensaje);
        String texto = TECLADO.nextLine().trim();
        if (!texto.isEmpty()) {
            return texto;
        }
        System.out.println(">> El dato no puede quedar vacio.");
    }
}

```

Paso 3. Estos tres métodos concentran toda la validación del programa:

- `leerEntero()` lee la línea completa e intenta convertirla con `Integer.parseInt()`. Si el usuario escribe texto, se captura `NumberFormatException`, se avisa y se vuelve a preguntar dentro del bucle `while(true)`. El programa nunca se interrumpe.
- `leerEnteroEnRango()` reutiliza el anterior y además obliga a que el valor este entre un mínimo y un máximo. Se usa para la opción del menú (0-7), la cantidad de prácticas (2-20) y cada nota (0-20).
- `leerTexto()` impide que un campo quede vacio.

Se usa `nextLine()` y no `nextInt()` precisamente para evitar el problema clásico del Scanner, que deja el salto de línea pendiente en el buffer y desordena las lecturas posteriores de texto.

```

/** Agrega un elemento al final del arreglo (usa getNumero, que es polimorfico). */
public void lectura() {
    if (!verificaCreado()) {
        return;
    }
    if (getTope() < getCapacidad()) {
        getNumerito()[getTope()] = getNumero();
        setTope(getTope() + 1);
    } else {
        System.out.println(">> El arreglo esta lleno.");
    }
}

/**
 * Lee UN numero. Las clases hijas pueden sobrescribirlo para aplicar sus
 * propias reglas de validacion (POLIMORFISMO DINAMICO).
 */
public int getNumero() {
    return leerEntero("Ingrese Numero : ");
}

```

Paso 4. Aquí esta la clave del diseño: `lectura()` guarda en la siguiente posición libre el valor que devuelva `getNumero()`. Como ese método puede ser sobrescrito por las clases hijas, el mismo código de almacenamiento sirve tanto para números cualquiera como para notas validadas. Eso es polimorfismo dinámico.

```

/** Metodo abstracto: cada clase hija lo implementa a su manera. */
public abstract void mostrarInformacion();
}

```

Paso 5. El método abstracto no tiene cuerpo. Obliga a cada clase hija a presentarse a su manera, y permite recorrer un arreglo de tipo `ArregloBase` llamando siempre al mismo método.

6.2 Arreglos y ArregloEstadistico: herencia y sobrecarga

```

public class Arreglos extends ArregloBase {

    public Arreglos() {

```

```

        super();
    }

    public Arreglos(int valor) {
        super(valor);
    }

    public Arreglos(ArregloBase otro) {
        super(otro);
    }

```

Paso 6. La palabra **extends** establece la herencia. Los constructores llaman a `super( . . . )`, es decir, reutilizan el constructor del padre en lugar de repetir su código.

```

// POLIMORFISMO ESTÁTICO (sobrecarga): mismo nombre, distintos parametros.
// Inserta x al final del arreglo.
public boolean insertar(int x) {
    return insertar(x, getTope() + 1);
}

```

Paso 7. Dos métodos comparten el nombre `insertar` pero reciben distinta cantidad de parámetros: el compilador decide cuál usar. Eso es polimorfismo estático o sobrecarga.

```

public interface Calculable {

    int sumar();

    int hallarMenor();

    int hallarMayor();

    double hallarPromedio();

    int sumarImpares();
}

```

Paso 8. La interfaz define el contrato de cálculo. Cualquier clase que la implemente esta obligada a escribir esos cinco métodos.

```

// 5a. Numero menor
@Override
public int hallarMenor() {
    exigeDatos();
    int menor = getNumerito()[0];
    for (int i = 1; i < getTope(); i++) {
        if (getNumerito()[i] < menor) {
            menor = getNumerito()[i];
        }
    }
    return menor;
}

// 5b. Numero mayor
@Override
public int hallarMayor() {
    exigeDatos();
    int mayor = getNumerito()[0];
    for (int i = 1; i < getTope(); i++) {
        if (getNumerito()[i] > mayor) {
            mayor = getNumerito()[i];
        }
    }
    return mayor;
}

```

Paso 9. `hallarMenor()` aplica el recorrido clásico: supone que el primer elemento es el menor y lo va reemplazando cada vez que encuentra uno más pequeño. Este método, heredado, es el que después permitirá eliminar la nota más baja sin escribir código nuevo.

6.3 ArregloNotas: la clase que resuelve la actividad

```
// Reglas del negocio (constantes encapsuladas)
public static final int NOTA_MINIMA = 0;
public static final int NOTA_MAXIMA = 20;
public static final int NOTA_APROBATORIA = 11;
```

Paso 10. Las reglas del negocio se guardan como constantes (static final). Si mañana la escala cambia, se modifica en un solo lugar y no en todo el programa.

```
// ----- POLIMORFISMO DINAMICO: validacion propia de la nota -----
/**
 * Sobrescribe el metodo de ArregloBase. El metodo lectura() de la clase
 * padre sigue siendo el mismo, pero ahora pide una NOTA validada entre
 * 0 y 20 en lugar de un numero entero cualquiera.
 */
@Override
public int getNumero() {
    return leerEnteroEnRango("Nota de la Practica " + (getTope() + 1)
        + " (" + NOTA_MINIMA + "-" + NOTA_MAXIMA + ") : ",
        NOTA_MINIMA, NOTA_MAXIMA);
}
```

Paso 11. Al sobrescribir getNumero(), el método lectura() heredado deja de pedir "un número" y pasa a pedir "la nota de la práctica N, entre 0 y 20", con su validación incluida. No se tocó ni una línea de la clase padre.

```
/** Ingresa todas las notas pendientes hasta llenar el arreglo. */
public void ingresarNotas() {
    if (!verificaCreado()) {
        return;
    }
    if (estaLleno()) {
        System.out.println(">> Todas las notas ya fueron registradas.");
        return;
    }
    System.out.println("Ingreso de notas (faltan " + (getCapacidad() - getTope()) + "):");
    while (!estaLleno()) {
        lectura();
    }
    System.out.println(">> Notas registradas correctamente.");
}
```

Paso 12. El bucle while (!estaLleno()) garantiza el requerimiento "ingresar las notas de todas las prácticas": no termina hasta que el arreglo este completo.

```
/** Posicion (0..tope-1) de la primera nota mas baja. */
public int posicionMenor() {
    int pos = 0;
    for (int i = 1; i < getTope(); i++) {
        if (getNumerito()[i] < getNumerito()[pos]) {
            pos = i;
        }
    }
    return pos;
}

/**
 * Promedio de las practicas ELIMINANDO la nota mas baja.
 * Se necesitan al menos 2 practicas para poder eliminar una.
 */
public double promedioSinMenor() {
    if (getTope() < 2) {
        throw new IllegalStateException("Se necesitan al menos 2 practicas para eliminar la nota mas baja.");
    }
    return (double) (sumar() - hallarMenor()) / (getTope() - 1);
}

/** Promedio final redondeado al entero mas cercano (0.5 sube). */
public int promedioRedondeado() {
    return (int) Math.round(promedioSinMenor());
}

// POLIMORFISMO ESTATICO (sobrecarga): mismo nombre, distintos parametros.
/** Promedio redondeado a la cantidad de decimales indicada. */
```

```

public double promedioRedondeado(int decimales) {
    if (decimales < 0) {
        throw new IllegalArgumentException("Los decimales no pueden ser negativos.");
    }
    double factor = Math.pow(10, decimales);
    return Math.round(promedioSinMenor() * factor) / factor;
}

```

Paso 13. Este es el núcleo de la actividad:

- `posicionMenor()` devuelve en que práctica se obtuvo la nota más baja, solo para informarlo en el reporte.
- `promedioSinMenor()` resta la nota menor a la suma total y divide entre (`tope - 1`). El casteo (`double`) evita que Java haga división entera y trunque los decimales. Si hay menos de 2 prácticas lanza una excepción controlada.
- `promedioRedondeado()` aplica `Math.round()` y entrega el entero pedido por el enunciado; `promedioRedondeado(int)` es su versión sobrecargada con decimales configurables.

```

/** Reporte completo del calculo pedido en la actividad. */
public void mostrarPromedioPracticas() {
    if (!verificaConDatos()) {
        return;
    }
    if (getTope() < 2) {
        System.out.println(">> Se necesitan al menos 2 practicas para eliminar la nota mas baja.");
        return;
    }

    int menor = hallarMenor();
    int pos = posicionMenor();

    System.out.println("-----");
    System.out.println(" CALCULO DEL PROMEDIO DE PRACTICAS");
    System.out.println("-----");
    System.out.println("Practicas registradas : " + getTope());
    System.out.println("Suma de todas las notas: " + sumar());
    System.out.println("Nota mas baja eliminada: " + menor + " (Practica " + (pos + 1) + ")");
    System.out.println("Suma sin la nota mas baja: " + (sumar() - menor));
    System.out.println("Practicas consideradas : " + (getTope() - 1));
    System.out.println("Promedio exacto      : "
        + String.format(Locale.US, "%.2f", promedioSinMenor()));
    System.out.println("PROMEDIO REDONDEADO   : " + promedioRedondeado());
    System.out.println("Condicion             : "
        + (estaAprobado() ? "APROBADO" : "DESAPROBADO"));
    System.out.println("-----");
}

```

Paso 14. El reporte no muestra solo el resultado: muestra la suma, la nota eliminada, cuantas prácticas quedaron y el promedio exacto antes del redondeo. Así el alumno puede verificar el cálculo a mano, que es justamente lo que se comprueba en la sección VIII.

```

// POLIMORFISMO DINAMICO: listado con formato de notas
@Override
public void escritura() {
    System.out.println("\tPRACTICA\tNOTA");
    System.out.println("\t-----\t----");
    if (!tieneDatos()) {
        System.out.println("\tNo hay notas registradas.");
        return;
    }
    for (int i = 0; i < getTope(); i++) {
        System.out.println("\tPractica " + (i + 1) + "\t" + getNumerito()[i]);
    }
    System.out.println("\tRegistradas: " + getTope() + " de " + getCapacidad());
}

```

Paso 15. Se sobrescribe `escritura()` para presentar los datos como una tabla Práctica / Nota en lugar de una fila de números sueltos.

6.4 Persona, Alumno y Docente

```

private String nombres;
private String apellidos;

```

```

public Persona(String nombres, String apellidos) {
    setNombres(nombres);
    setApellidos(apellidos);
}

public String getNombres() {
    return nombres;
}

public final void setNombres(String nombres) {
    if (nombres == null || nombres.trim().isEmpty()) {
        throw new IllegalArgumentException("Los nombres no pueden estar vacios.");
    }
    this.nombres = nombres.trim();
}

public String getApellidos() {
    return apellidos;
}

public final void setApellidos(String apellidos) {
    if (apellidos == null || apellidos.trim().isEmpty()) {
        throw new IllegalArgumentException("Los apellidos no pueden estar vacios.");
    }
    this.apellidos = apellidos.trim();
}

```

Paso 16. El constructor no asigna directamente: llama a los setters, de manera que la validación se aplica también al crear el objeto. Los setters se declaran **final** para que ninguna clase hija pueda anular esa validación.

```

public class Alumno extends Persona {

    private String codigo;
    private String curso;
    private ArregloNotas practicas;

    public Alumno(String nombres, String apellidos, String codigo, String curso) {
        super(nombres, apellidos);
        setCodigo(codigo);
        setCurso(curso);
        this.practicas = new ArregloNotas();
    }
}

```

Paso 17. Además de heredar de Persona, el Alumno **contiene** un objeto ArregloNotas. A esta relación "tiene un" se le llama composición y convive perfectamente con la herencia "es un".

```

/** Crea el arreglo donde se guardaran las notas de las practicas. */
public void registrarCantidadPracticas(int cantidad) {
    if (cantidad < 2) {
        throw new IllegalArgumentException("Debe haber al menos 2 practicas para poder eliminar la mas baja.");
    }
    this.practicas = new ArregloNotas(cantidad);
}
}

```

Paso 18. Se exige un mínimo de 2 prácticas desde el propio objeto, no solo desde el menú. La regla vive donde viven los datos.

6.5 AplicaNotas: el menú y la demostración de POO

```

/** Verifica que exista un alumno antes de operar. */
static boolean hayAlumno() {
    if (alumno == null) {
        System.out.println(">> Primero debe registrar los datos del alumno (opcion 1).");
        return false;
    }
    return true;
}

/** VALIDACION: ningun dato puede quedar vacio. */
static void registrarAlumno() {
    String nombres = ArregloBase.leerTexto("Nombres del alumno : ");
    String apellidos = ArregloBase.leerTexto("Apellidos del alumno : ");
    String codigo = ArregloBase.leerTexto("Codigo del alumno : ");
    String curso = ArregloBase.leerTexto("Curso : ");

    alumno = new Alumno(nombres, apellidos, codigo, curso);
}

```

```
        System.out.println(">> Alumno registrado: " + alumno.getNombreCompleto());  
    }
```

Paso 19. Antes de cualquier operación se comprueba que el alumno exista y que ningún campo quede vacío. El menú principal también usa `LeerEnteroEnRango(0, 7)`, de modo que ni siquiera es necesario un caso `default` para opciones inválidas.

```
// 1) POLIMORFISMO con la jerarquia Persona  
System.out.println("-- 1. Polimorfismo con Persona[] --");  
Docente docente = new Docente("Carlos", "Mendoza Rojas", "Algoritmica");  
Persona[] personas = (alumno == null)  
    ? new Persona[]{docente}  
    : new Persona[]{alumno, docente};  
  
for (Persona p : personas) {  
    System.out.println("-> " + p);    // usa toString() sobrescrito  
    p.mostrarDatos();                // cada clase responde distinto  
}  
  
// 2) POLIMORFISMO con la jerarquia de arreglos  
System.out.println("-- 2. Polimorfismo con ArregloBase[] --");  
ArregloBase[] estructuras = (alumno == null)  
    ? new ArregloBase[]{new Arreglos(), new ArregloEstadistico()}  
    : new ArregloBase[]{new Arreglos(alumno.getPracticas()),  
        new ArregloEstadistico(), alumno.getPracticas()};  
  
for (ArregloBase a : estructuras) {  
    a.mostrarInformacion();          // metodo abstracto implementado  
}
```

Paso 20. Estos dos bucles son la prueba visible del polimorfismo: una misma variable de tipo padre recorre objetos de clases distintas y cada uno responde con su propia implementación. El resultado de ejecutarlo aparece en la Evidencia 7.

VII. Evidencias de ejecución

Todas las capturas siguientes corresponden a ejecuciones reales del programa desde la consola de NetBeans. Los valores que aparecen después de los dos puntos son los datos escritos por el usuario.

Caso de prueba 1 - Alumno: Quispe Ramos, Luis Enrique | 5 prácticas | Notas: 14, 8, 17, 12, 15.

Evidencia 1. Menú principal del sistema

```

=====
MENU DE OPCIONES
=====
1.- Registrar datos del alumno
2.- Registrar cantidad de practicas
3.- Ingresar notas de las practicas
=====
4.- Mostrar notas registradas
5.- Calcular promedio (sin la nota mas baja)
6.- Ficha completa del alumno
=====
7.- Demostracion de POO (herencia y polimorfismo)
0.- Salir
=====

```

El menú agrupa las opciones en tres bloques: registro, consulta y demostración académica. La lectura de la alternativa ya viene validada entre 0 y 7.

Evidencia 2. Registro del alumno y validación de campos vacios

```

Ingrese una alternativa (0-7) : 1
Nombres del alumno :
>> El dato no puede quedar vacio.
Nombres del alumno : Luis Enrique
Apellidos del alumno : Quispe Ramos
Codigo del alumno : 2024100345
Curso : Algoritmica y Programacion
>> Alumno registrado: Quispe Ramos, Luis Enrique

```

Se presionó Enter sin escribir nada en el primer campo. El programa lo detectó, mostró el aviso y volvió a solicitar el dato en lugar de continuar con un nombre en blanco.

Evidencia 3. Creación del arreglo y validación de tipo y rango

```

Ingrese una alternativa (0-7) : 2
Cantidad de practicas del curso (2-20) : cinco
>> Dato no valido: "cinco". Ingrese un numero entero.
Cantidad de practicas del curso (2-20) : 1
>> Fuera de rango. El valor debe estar entre 2 y 20.
Cantidad de practicas del curso (2-20) : 5
>> Se creo el arreglo para 5 practicas.

```

Se probaron dos errores a propósito. Primero se escribió la palabra "cinco": el programa capturo la excepción y pidio un número entero. Luego se ingreso 1, que es menor al mínimo permitido, y se informó el rango correcto. Recien con el valor 5 se creó el arreglo.

Evidencia 4. Ingreso de las notas con validación 0-20

```

Ingrese una alternativa (0-7) : 3
Ingreso de notas (faltan 5):
Nota de la Practica 1 (0-20) : quince
>> Dato no valido: "quince". Ingrese un numero entero.
Nota de la Practica 1 (0-20) : 25
>> Fuera de rango. El valor debe estar entre 0 y 20.
Nota de la Practica 1 (0-20) : 14
Nota de la Practica 2 (0-20) : 8
Nota de la Practica 3 (0-20) : 17
Nota de la Practica 4 (0-20) : 12
Nota de la Practica 5 (0-20) : 15
>> Notas registradas correctamente.

```

Nuevamente se probaron entradas inválidas: el texto "quince" y la nota 25, superior al máximo. En ambos casos se repitió la petición de la *misma* práctica, sin avanzar ni perder datos, hasta que se ingreso un valor correcto.

Evidencia 5. Listado de las notas almacenadas

```

Ingrese una alternativa (0-7) : 4
PRACTICA    NOTA
-----
Practica 1   14
Practica 2   8
Practica 3   17
Practica 4   12
Practica 5   15
Registradas: 5 de 5

```

Se confirma que las cinco notas quedaron almacenadas en el arreglo, cada una asociada al número de práctica que le corresponde.

Evidencia 6. Cálculo del promedio eliminando la nota más baja

```

Ingrese una alternativa (0-7) : 5
-----
CALCULO DEL PROMEDIO DE PRACTICAS
-----
Practicas registradas : 5
Suma de todas las notas: 66
Nota mas baja eliminada: 8 (Practica 2)
Suma sin la nota mas baja: 58
Practicas consideradas : 4
Promedio exacto       : 14.50
PROMEDIO REDONDEADO   : 15
Condicion              : APROBADO
-----

```

Resultado central de la actividad. El programa identificó la nota más baja (8, correspondiente a la práctica 2), la excluyó del cálculo, promedió las cuatro notas restantes obteniendo 14.50 y mostró el **promedio redondeado: 15**.

Evidencia 7. Ficha completa del alumno

```

Ingrese una alternativa (0-7) : 6
=====
FICHA DEL ALUMNO
=====
Alumno : Quispe Ramos, Luis Enrique
Codigo : 2024100345
Curso  : Algoritmica y Programacion
-----
PRACTICA    NOTA
-----
Practica 1   14
Practica 2   8
Practica 3   17
Practica 4   12
Practica 5   15
Registradas: 5 de 5
-----
Nota mas baja eliminada : 8
PROMEDIO REDONDEADO     : 15
Condicion                : APROBADO
=====

```

La ficha reúne los datos personales, el detalle de las prácticas y el resultado final. Es la salida que conviene mostrar como resumen del trabajo.

Evidencia 8. Demostración de los pilares de la POO en ejecución

```

Ingrese una alternativa (0-7) : 7
=====
DEMOSTRACION DE PROGRAMACION ORIENTADA A OBJETOS
=====
-- 1. Polimorfismo con Persona[] --
-> Alumno: Quispe Ramos, Luis Enrique
=====
FICHA DEL ALUMNO
=====
Alumno : Quispe Ramos, Luis Enrique
Codigo : 2024100345
Curso : Algoritmica y Programacion
=====
PRACTICA    NOTA
-----
Practica 1   14
Practica 2    8
Practica 3   17
Practica 4   12
Practica 5   15
Registradas: 5 de 5
=====
Nota mas baja eliminada : 8
PROMEDIO REDONDEADO      : 15
Condicion                 : APROBADO
=====
-> Docente: Mendoza Rojas, Carlos
Docente del curso : Mendoza Rojas, Carlos | Especialidad: Algoritmica
-- 2. Polimorfismo con ArregloBase[] --
[Arreglos] Operaciones de edicion: eliminar, modificar, insertar y mover elementos.
[ArregloEstadistico] Hereda de Arreglos y agrega: suma, menor, mayor, promedio y suma de impares.
[ArregloNotas] Arreglo de notas 0-20. Calcula el promedio eliminando la nota mas baja y lo muestra
redondeado.
-- 3. Sobrecarga de metodos --
promedioRedondeado() -> 15
promedioRedondeado(2) -> 14.50
-- 4. Encapsulamiento (el setter rechaza datos invalidos) --
Excepcion controlada: Tope fuera de rango: 99
=====

```

Esta salida evidencia los cuatro pilares funcionando:

- **Polimorfismo por referencia:** el arreglo `Persona[]` recorre un Alumno y un Docente; el mismo llamado a `mostrarDatos()` produce una ficha completa en un caso y una sola línea en el otro.
- **Herencia y abstracción:** el arreglo `ArregloBase[]` contiene tres objetos de clases distintas de la misma cadena de herencia, y cada uno responde el método abstracto `mostrarInformacion()` con su propia descripción.
- **Sobrecarga:** `promedioRedondeado()` devuelve 15 y `promedioRedondeado(2)` devuelve 14.50, con el mismo nombre de método.
- **Encapsulamiento:** al intentar asignar un tope de 99 a un arreglo de capacidad 3, el setter lanzó la excepción y el objeto mantuvo su estado válido.

Evidencia 9. Operaciones bloqueadas por falta de datos previos

```

Ingrese una alternativa (0-7) : 5
>> Primero debe registrar los datos del alumno (opcion 1).
Ingrese una alternativa (0-7) : 3
>> Primero debe registrar los datos del alumno (opcion 1).

```

Al intentar calcular el promedio e ingresar notas sin haber registrado al alumno, el programa no se detuvo ni generó un `NullPointerException`: informó el paso que faltaba y volvió al menú.

Evidencia 10. Segundo caso de prueba (redondeo hacia abajo)

```

Ingrese una alternativa (0-7) : 3
Ingreso de notas (faltan 4):
Nota de la Practica 1 (0-20) : 11
Nota de la Practica 2 (0-20) : 12
Nota de la Practica 3 (0-20) : 10
Nota de la Practica 4 (0-20) : 14
>> Notas registradas correctamente.
Ingrese una alternativa (0-7) : 5
=====

```

```
CALCULO DEL PROMEDIO DE PRACTICAS
-----
Practicas registradas : 4
Suma de todas las notas: 47
Nota mas baja eliminada: 10 (Practica 3)
Suma sin la nota mas baja: 37
Practicas consideradas : 3
Promedio exacto       : 12.33
PROMEDIO REDONDEADO   : 12
Condicion              : APROBADO
-----
```

Con las notas 11, 12, 10 y 14 se elimina la nota 10 y el promedio exacto es 12.33, que al redondearse baja a **12**. Junto con el caso anterior (14.50 que sube a 15) se demuestra que el redondeo funciona en ambos sentidos.

VIII. Resultados y verificación de los cálculos

Para comprobar que el programa calcula correctamente, cada caso se resolvió también de forma manual con la fórmula de la sección 3.4.

Caso	Notas ingresadas	Suma	Nota más baja	Suma - menor	n - 1	Promedio exacto	Redondeado	Salida del programa
1	14, 8, 17, 12, 15	66	8 (Práctica 2)	58	4	14.50	15	15 - APROBADO
2	11, 12, 10, 14	47	10 (Práctica 3)	37	3	12.33	12	12 - APROBADO

En ambos casos el valor calculado a mano coincide exactamente con el mostrado por el programa, lo que valida la implementación.

8.1 Resumen de las pruebas de validación

N	Prueba realizada	Dato ingresado	Resultado esperado	Resultado obtenido
P1	Campo de texto vacío	(Enter)	Solicitar nuevamente el dato	Correcto
P2	Texto donde va un número	cinco	Avisar y volver a pedir	Correcto
P3	Cantidad de prácticas fuera de rango	1	Exigir un valor entre 2 y 20	Correcto
P4	Nota no numérica	quince	Avisar y repetir la misma práctica	Correcto
P5	Nota fuera de la escala	25	Exigir un valor entre 0 y 20	Correcto
P6	Calcular sin alumno registrado	Opción 5	Indicar que falta el paso 1	Correcto
P7	Ingresar notas sin alumno registrado	Opción 3	Indicar que falta el paso 1	Correcto
P8	Tope inválido por código	setTope(99)	Lanzar IllegalArgumentException	Correcto

IX. Conclusiones

- Se cumplió íntegramente el enunciado: el programa ingresa y almacena las notas de todas las prácticas, elimina la nota más baja, calcula el promedio de las restantes y lo muestra redondeado, validando cada dato ingresado.
- La herencia multinivel permitió reutilizar código de manera efectiva: métodos como `sumar()` y `hallarMenor()`, escritos una sola vez en `ArregloEstadistico`, son los que hacen posible el cálculo del promedio sin la nota más baja en `ArregloNotas`.
- El polimorfismo dinámico evitó duplicar lógica: al sobrescribir únicamente `getNumero()`, todo el mecanismo de almacenamiento heredado pasó a trabajar con notas validadas, sin modificar la clase padre.
- El encapsulamiento demostró su utilidad práctica: al validar dentro de los setters, el objeto se protege incluso de errores cometidos desde otras clases y no solo de errores del usuario.
- La abstracción, mediante la clase abstracta `ArregloBase` y la interfaz `Calculable`, permitió separar que debe hacer cada estructura de como lo hace, facilitando agregar nuevas clases en el futuro.
- La validación de datos resultó tan importante como el cálculo: las pruebas P1 a P8 confirman que el programa no se interrumpe ante entradas incorrectas, requisito indispensable en una aplicación de uso real.

XI. Anexo A: código fuente completo

Proyecto **Arreglo**, paquete **Arreglo**. Para ejecutarlo en NetBeans: clic derecho sobre el proyecto, Properties, Run, y establecer **Arreglo.AplicaNotas** como Main Class. Los archivos se listan en el orden en que se recomienda leerlos.

A.1 Calculable.java

Interfaz: contrato de los cálculos (abstracción).

```
package Arreglo;

/**
 * ABSTRACCION (interfaz): define QUE calculos debe ofrecer un arreglo
 * estadistico, sin decir COMO se calculan. Cada clase que la implemente
 * decide los detalles.
 */
public interface Calculable {

    int sumar();

    int hallarMenor();

    int hallarMayor();

    double hallarPromedio();

    int sumarImpares();
}
```

A.2 ArregloBase.java

Clase abstracta: estructura, encapsulamiento y validación.

```
package Arreglo;

import java.util.Arrays;
import java.util.Scanner;

/**
 * ABSTRACCION: clase abstracta que representa "un arreglo de enteros con tope".
 * No se puede instanciar directamente; obliga a las clases hijas a implementar
 * mostrarInformacion().
 *
 * ENCAPSULAMIENTO: los atributos son private y solo se accede a ellos
 * mediante getters y setters (con validacion).
 */
public abstract class ArregloBase {

    // Un solo Scanner compartido por todo el programa
    public static final Scanner TECLADO = new Scanner(System.in);

    // Atributos encapsulados
    private int[] numerito;
    private int tope;

    // Constructor por defecto
    public ArregloBase() {
        this.numerito = null;
        this.tope = 0;
    }

    // Constructor con parametro
    public ArregloBase(int valor) {
        if (valor < 1) {
            throw new IllegalArgumentException("El tamano debe ser mayor que 0.");
        }
        this.numerito = new int[valor];
        this.tope = 0;
    }

    // Constructor copia (lo usan las clases hijas)
    protected ArregloBase(ArregloBase otro) {
        if (otro.numerito == null) {
            this.numerito = null;
            this.tope = 0;
        } else {
            this.numerito = Arrays.copyOf(otro.numerito, otro.numerito.length);
        }
    }
}
```

```
        this.tope = otro.tope;
    }
}

// ----- Getters y Setters (ENCAPSULAMIENTO) -----
public int[] getNumerito() {
    return numerito;
}

public void setNumerito(int[] numerito) {
    this.numerito = numerito;
}

public int getTope() {
    return tope;
}

public void setTope(int tope) {
    int capacidad = (numerito == null) ? 0 : numerito.length;
    if (tope < 0 || tope > capacidad) {
        throw new IllegalArgumentException("Tope fuera de rango: " + tope);
    }
    this.tope = tope;
}

public int getCapacidad() {
    return (numerito == null) ? 0 : numerito.length;
}

// Metodo para crear el arreglo
public void creaArreglo(int valor) {
    if (valor < 1) {
        throw new IllegalArgumentException("El tamano debe ser mayor que 0.");
    }
    setNumerito(new int[valor]);
    setTope(0);
}

// ----- Metodos de consulta (no imprimen nada) -----
public boolean estaCreado() {
    return numerito != null;
}

public boolean tieneDatos() {
    return numerito != null && tope > 0;
}

public boolean estaLleno() {
    return numerito != null && tope == numerito.length;
}

/** Devuelve la posicion (0..tope-1) de la primera aparicion de x, o -1. */
protected int buscar(int x) {
    for (int i = 0; i < tope; i++) {
        if (numerito[i] == x) {
            return i;
        }
    }
    return -1;
}

// ----- Verificaciones que informan al usuario -----
public boolean verificaCreado() {
    if (!estaCreado()) {
        System.out.println(">> Primero debe crear el arreglo de practicas (opcion 2).");
        return false;
    }
    return true;
}

public boolean verificaConDatos() {
    if (!verificaCreado()) {
        return false;
    }
    if (tope == 0) {
        System.out.println(">> El arreglo no tiene elementos registrados.");
        return false;
    }
    return true;
}

public boolean verificaConEspacio() {
    if (!verificaCreado()) {
```

```

        return false;
    }
    if (estalleno()) {
        System.out.println(">> El arreglo esta lleno, no se puede insertar.");
        return false;
    }
    return true;
}

// ----- VALIDACION DE ENTRADA DE DATOS -----
/**
 * Lee una linea completa y la convierte a entero. Si el usuario escribe
 * letras, simbolos o deja vacio, vuelve a preguntar (no se cae el programa).
 */
public static int leerEntero(String mensaje) {
    while (true) {
        System.out.print(mensaje);
        String linea = TECLADO.nextLine().trim();
        try {
            return Integer.parseInt(linea);
        } catch (NumberFormatException e) {
            System.out.println(">> Dato no valido: \"" + linea + "\". Ingrese un numero entero.");
        }
    }
}

/** Lee un entero y ademas valida que este dentro del rango [min, max]. */
public static int leerEnteroEnRango(String mensaje, int min, int max) {
    while (true) {
        int valor = leerEntero(mensaje);
        if (valor >= min && valor <= max) {
            return valor;
        }
        System.out.println(">> Fuera de rango. El valor debe estar entre " + min + " y " + max + ".");
    }
}

/** Lee un texto no vacio. */
public static String leerTexto(String mensaje) {
    while (true) {
        System.out.print(mensaje);
        String texto = TECLADO.nextLine().trim();
        if (!texto.isEmpty()) {
            return texto;
        }
        System.out.println(">> El dato no puede quedar vacio.");
    }
}

// ----- Entrada de datos -----
/** Agrega un elemento al final del arreglo (usa getNumero, que es polimorfico). */
public void lectura() {
    if (!verificaCreado()) {
        return;
    }
    if (getTope() < getCapacidad()) {
        getNumerito()[getTope()] = getNumero();
        setTope(getTope() + 1);
    } else {
        System.out.println(">> El arreglo esta lleno.");
    }
}

/**
 * Lee UN numero. Las clases hijas pueden sobrescribirlo para aplicar sus
 * propias reglas de validacion (POLIMORFISMO DINAMICO).
 */
public int getNumero() {
    return leerEntero("Ingrese Numero : ");
}

// Metodo para mostrar datos (las hijas pueden sobrescribirlo)
public void escritura() {
    System.out.println("\tListado");
    System.out.println("\t-----");

    if (!tieneDatos()) {
        System.out.println("No hay elementos");
    } else {
        for (int i = 0; i < getTope(); i++) {
            System.out.print(getNumerito()[i] + "\t");
        }
        System.out.println();
    }
}

```

```

    }
}

/** Metodo abstracto: cada clase hija lo implementa a su manera. */
public abstract void mostrarInformacion();
}

```

A.3 Arreglos.java

Herencia nivel 1: operaciones que modifican el arreglo.

```

package Arreglo;

/**
 * HERENCIA: Arreglos extiende ArregloBase y reutiliza sus atributos y metodos.
 * Aquí van las operaciones que MODIFICAN el arreglo (actividades 1, 2, 3, 7 y 8).
 */
public class Arreglos extends ArregloBase {

    public Arreglos() {
        super();
    }

    public Arreglos(int valor) {
        super(valor);
    }

    public Arreglos(ArregloBase otro) {
        super(otro);
    }

    // 1. Eliminar un elemento "x"
    public boolean eliminar(int x) {
        if (!verificaConDatos()) {
            return false;
        }

        int pos = buscar(x);
        if (pos == -1) {
            System.out.println("El elemento " + x + " no se encuentra en el arreglo.");
            return false;
        }

        int[] a = getNumerito();
        for (int i = pos; i < getTope() - 1; i++) {
            a[i] = a[i + 1]; // se corren los elementos a la izquierda
        }
        a[getTope() - 1] = 0;
        setTope(getTope() - 1);
        System.out.println("Elemento " + x + " eliminado correctamente.");
        return true;
    }

    // 2. Modificar un elemento "x" por un valor nuevo
    public boolean modificar(int x, int nuevo) {
        if (!verificaConDatos()) {
            return false;
        }

        int pos = buscar(x);
        if (pos == -1) {
            System.out.println("El elemento " + x + " no se encuentra en el arreglo.");
            return false;
        }

        getNumerito()[pos] = nuevo;
        System.out.println("Elemento " + x + " modificado por " + nuevo + ".");
        return true;
    }

    // 3. Insertar un elemento "x" en la posicion "y"
    public boolean insertar(int x, int y) {
        if (!verificaConEspacio()) {
            return false;
        }
        if (y < 1 || y > getTope() + 1) {
            System.out.println("Posicion invalida. Debe estar entre 1 y " + (getTope() + 1) + ".");
            return false;
        }

        int[] a = getNumerito();
        for (int i = getTope(); i > y - 1; i--) {
            a[i] = a[i - 1]; // se corren los elementos a la derecha
        }
        a[y - 1] = x;
        setTope(getTope() + 1);
        System.out.println("Elemento " + x + " insertado en la posicion " + y + ".");
        return true;
    }
}

```

```

    }
    a[y - 1] = x;
    setTope(getTope() + 1);
    System.out.println("Elemento " + x + " insertado en la posicion " + y + ".");
    return true;
}

// POLIMORFISMO ESTATICO (sobrecarga): mismo nombre, distintos parametros.
// Inserta x al final del arreglo.
public boolean insertar(int x) {
    return insertar(x, getTope() + 1);
}

// 7. Mover el primer elemento a la ultima posicion
public void moverPrimeroAlUltimo() {
    if (!verificaConDatos()) {
        return;
    }

    int[] a = getNumerito();
    int primero = a[0];
    for (int i = 0; i < getTope() - 1; i++) {
        a[i] = a[i + 1];
    }
    a[getTope() - 1] = primero;
    System.out.println("El primer elemento (" + primero + ") paso a la ultima posicion.");
}

// 8. Mover el primer elemento par al final del arreglo
public void moverPrimerParAlFinal() {
    if (!verificaConDatos()) {
        return;
    }

    int[] a = getNumerito();
    int pos = -1;
    for (int i = 0; i < getTope(); i++) {
        if (a[i] % 2 == 0) {
            pos = i;
            break;
        }
    }

    if (pos == -1) {
        System.out.println("No hay elementos pares en el arreglo.");
        return;
    }

    int par = a[pos];
    for (int i = pos; i < getTope() - 1; i++) {
        a[i] = a[i + 1];
    }
    a[getTope() - 1] = par;
    System.out.println("El primer par (" + par + ") paso al final del arreglo.");
}

// POLIMORFISMO DINAMICO (sobrescritura): implementacion del metodo abstracto
@Override
public void mostrarInformacion() {
    System.out.println("[Arreglos] Operaciones de edicion: eliminar, modificar, insertar y mover elementos.");
}
}

```

A.4 ArregloEstadistico.java

Herencia nivel 2: cálculos estadísticos.

```

package Arreglo;

import java.util.Locale;

/**
 * HERENCIA (multinivel): ArregloEstadistico -> Arreglos -> ArregloBase.
 * ABSTRACCION: implementa la interfaz Calculable.
 * Aquí van las operaciones de CALCULO (actividades 4, 5 y 6).
 */
public class ArregloEstadistico extends Arreglos implements Calculable {

    public ArregloEstadistico() {
        super();
    }

    public ArregloEstadistico(int valor) {

```

```
        super(valor);
    }

    private void exigeDatos() {
        if (!tieneDatos()) {
            throw new IllegalStateException("El arreglo no tiene elementos.");
        }
    }

    // 4. Sumar los elementos del arreglo
    @Override
    public int sumar() {
        exigeDatos();
        int suma = 0;
        for (int i = 0; i < getTope(); i++) {
            suma += getNumerito()[i];
        }
        return suma;
    }

    // 5a. Numero menor
    @Override
    public int hallarMenor() {
        exigeDatos();
        int menor = getNumerito()[0];
        for (int i = 1; i < getTope(); i++) {
            if (getNumerito()[i] < menor) {
                menor = getNumerito()[i];
            }
        }
        return menor;
    }

    // 5b. Numero mayor
    @Override
    public int hallarMayor() {
        exigeDatos();
        int mayor = getNumerito()[0];
        for (int i = 1; i < getTope(); i++) {
            if (getNumerito()[i] > mayor) {
                mayor = getNumerito()[i];
            }
        }
        return mayor;
    }

    // 5c. Promedio
    @Override
    public double hallarPromedio() {
        return (double) sumar() / getTope();
    }

    // 6. Suma de los elementos impares
    @Override
    public int sumarImpares() {
        exigeDatos();
        int suma = 0;
        for (int i = 0; i < getTope(); i++) {
            if (getNumerito()[i] % 2 != 0) { // funciona tambien con negativos
                suma += getNumerito()[i];
            }
        }
        return suma;
    }

    // ----- Metodos que muestran los resultados -----
    public void mostrarSuma() {
        if (!verificaConDatos()) {
            return;
        }
        System.out.println("La suma de los elementos es: " + sumar());
    }

    public void mostrarEstadisticas() {
        if (!verificaConDatos()) {
            return;
        }
        System.out.println("Numero menor : " + hallarMenor());
        System.out.println("Numero mayor : " + hallarMayor());
        System.out.println("Promedio      : " + String.format(Locale.US, "%.2f", hallarPromedio()));
    }

    public void mostrarSumaImpares() {
```

```

        if (!verificaConDatos()) {
            return;
        }
        System.out.println("La suma de los elementos impares es: " + sumarImpares());
    }

    // POLIMORFISMO DINAMICO: sobrescribe el metodo de la clase padre
    @Override
    public void mostrarInformacion() {
        System.out.println("[ArregloEstadistico] Hereda de Arreglos y agrega: suma, menor, mayor, promedio y suma de impares.");
    }

    // POLIMORFISMO DINAMICO: amplia el listado heredado
    @Override
    public void escritura() {
        super.escritura();
        if (estaCreado()) {
            System.out.println("Elementos: " + getTope() + " de " + getNumerito().length);
        }
    }
}

```

A.5 ArregloNotas.java

Herencia nivel 3: notas 0-20 y promedio sin la más baja.

```

package Arreglo;

import java.util.Locale;

/**
 * HERENCIA MULTINIVEL:
 * ArregloNotas -> ArregloEstadistico -> Arreglos -> ArregloBase
 *
 * Esta clase especializa el arreglo generico de enteros para que se comporte
 * como un "arreglo de notas de practicas": valida el rango 0-20 y calcula el
 * promedio eliminando la nota mas baja.
 */
public class ArregloNotas extends ArregloEstadistico {

    // Reglas del negocio (constantes encapsuladas)
    public static final int NOTA_MINIMA = 0;
    public static final int NOTA_MAXIMA = 20;
    public static final int NOTA_APROBATORIA = 11;

    public ArregloNotas() {
        super();
    }

    public ArregloNotas(int cantidadPracticas) {
        super(cantidadPracticas);
    }

    // ----- POLIMORFISMO DINAMICO: validacion propia de la nota -----
    /**
     * Sobrescribe el metodo de ArregloBase. El metodo lectura() de la clase
     * padre sigue siendo el mismo, pero ahora pide una NOTA validada entre
     * 0 y 20 en lugar de un numero entero cualquiera.
     */
    @Override
    public int getNumero() {
        return leerEnteroEnRango("Nota de la Practica " + (getTope() + 1)
            + " (" + NOTA_MINIMA + "-" + NOTA_MAXIMA + ") : ",
            NOTA_MINIMA, NOTA_MAXIMA);
    }

    /** Ingresa todas las notas pendientes hasta llenar el arreglo. */
    public void ingresarNotas() {
        if (!verificaCreado()) {
            return;
        }
        if (estaLleno()) {
            System.out.println(">> Todas las notas ya fueron registradas.");
            return;
        }
        System.out.println("Ingreso de notas (faltan " + (getCapacidad() - getTope()) + "):");
        while (!estaLleno()) {
            lectura();
        }
        System.out.println(">> Notas registradas correctamente.");
    }

    // ----- Calculos propios de la actividad -----

```

```

/** Posicion (0..tope-1) de la primera nota mas baja. */
public int posicionMenor() {
    int pos = 0;
    for (int i = 1; i < getTope(); i++) {
        if (getNumerito()[i] < getNumerito()[pos]) {
            pos = i;
        }
    }
    return pos;
}

/**
 * Promedio de las practicas ELIMINANDO la nota mas baja.
 * Se necesitan al menos 2 practicas para poder eliminar una.
 */
public double promedioSinMenor() {
    if (getTope() < 2) {
        throw new IllegalStateException("Se necesitan al menos 2 practicas para eliminar la nota mas
            baja.");
    }
    return (double) (sumar() - hallarMenor()) / (getTope() - 1);
}

/** Promedio final redondeado al entero mas cercano (0.5 sube). */
public int promedioRedondeado() {
    return (int) Math.round(promedioSinMenor());
}

// POLIMORFISMO ESTATICO (sobrecarga): mismo nombre, distintos parametros.
/** Promedio redondeado a la cantidad de decimales indicada. */
public double promedioRedondeado(int decimales) {
    if (decimales < 0) {
        throw new IllegalArgumentException("Los decimales no pueden ser negativos.");
    }
    double factor = Math.pow(10, decimales);
    return Math.round(promedioSinMenor() * factor) / factor;
}

public boolean estaAprobado() {
    return promedioRedondeado() >= NOTA_APROBATORIA;
}

// ----- Salidas por pantalla -----
/** Reporte completo del calculo pedido en la actividad. */
public void mostrarPromedioPracticas() {
    if (!verificaConDatos()) {
        return;
    }
    if (getTope() < 2) {
        System.out.println(">> Se necesitan al menos 2 practicas para eliminar la nota mas baja.");
        return;
    }

    int menor = hallarMenor();
    int pos = posicionMenor();

    System.out.println("-----");
    System.out.println(" CALCULO DEL PROMEDIO DE PRACTICAS");
    System.out.println("-----");
    System.out.println("Practicas registradas : " + getTope());
    System.out.println("Suma de todas las notas: " + sumar());
    System.out.println("Nota mas baja eliminada: " + menor + " (Practica " + (pos + 1) + ")");
    System.out.println("Suma sin la nota mas baja: " + (sumar() - menor));
    System.out.println("Practicas consideradas : " + (getTope() - 1));
    System.out.println("Promedio exacto      : "
        + String.format(Locale.US, "%.2f", promedioSinMenor()));
    System.out.println("PROMEDIO REDONDEADO   : " + promedioRedondeado());
    System.out.println("Condicion             : "
        + (estaAprobado() ? "APROBADO" : "DESAPROBADO"));
    System.out.println("-----");
}

// POLIMORFISMO DINAMICO: listado con formato de notas
@Override
public void escritura() {
    System.out.println("\tPRACTICA\tNOTA");
    System.out.println("\t-----\t----");
    if (!tieneDatos()) {
        System.out.println("\tNo hay notas registradas.");
        return;
    }
    for (int i = 0; i < getTope(); i++) {
        System.out.println("\tPractica " + (i + 1) + "\t" + getNumerito()[i]);
    }
}

```

```

    }
    System.out.println("\tRegistradas: " + getTope() + " de " + getCapacidad());
}

// POLIMORFISMO DINAMICO: implementacion propia del metodo abstracto
@Override
public void mostrarInformacion() {
    System.out.println("[ArregloNotas] Arreglo de notas 0-20. Calcula el promedio "
        + "eliminando la nota mas baja y lo muestra redondeado.");
}
}

```

A.6 Persona.java

Clase abstracta de la segunda jerarquía.

```

package Arreglo;

/**
 * ABSTRACCION: modela lo comun a cualquier persona del sistema academico.
 * No tiene sentido crear una "Persona" suelta, por eso es abstracta.
 *
 * ENCAPSULAMIENTO: nombres y apellidos son private y se validan en los setters.
 */
public abstract class Persona {

    private String nombres;
    private String apellidos;

    public Persona(String nombres, String apellidos) {
        setNombres(nombres);
        setApellidos(apellidos);
    }

    public String getNombres() {
        return nombres;
    }

    public final void setNombres(String nombres) {
        if (nombres == null || nombres.trim().isEmpty()) {
            throw new IllegalArgumentException("Los nombres no pueden estar vacios.");
        }
        this.nombres = nombres.trim();
    }

    public String getApellidos() {
        return apellidos;
    }

    public final void setApellidos(String apellidos) {
        if (apellidos == null || apellidos.trim().isEmpty()) {
            throw new IllegalArgumentException("Los apellidos no pueden estar vacios.");
        }
        this.apellidos = apellidos.trim();
    }

    public String getNombreCompleto() {
        return apellidos + ", " + nombres;
    }

    /** Cada tipo de persona dice que rol cumple. */
    public abstract String getRol();

    /** Cada tipo de persona se muestra a su manera (POLIMORFISMO). */
    public abstract void mostrarDatos();

    @Override
    public String toString() {
        return getRol() + ": " + getNombreCompleto();
    }
}

```

A.7 Alumno.java

Hereda de Persona y contiene un ArregloNotas.

```

package Arreglo;

/**
 * HERENCIA: Alumno es una Persona.
 * COMPOSICION: un Alumno TIENE un ArregloNotas con sus practicas.

```

```
*/
public class Alumno extends Persona {

    private String codigo;
    private String curso;
    private ArregloNotas practicas;

    public Alumno(String nombres, String apellidos, String codigo, String curso) {
        super(nombres, apellidos);
        setCodigo(codigo);
        setCurso(curso);
        this.practicas = new ArregloNotas();
    }

    public String getCodigo() {
        return codigo;
    }

    public final void setCodigo(String codigo) {
        if (codigo == null || codigo.trim().isEmpty()) {
            throw new IllegalArgumentException("El codigo no puede estar vacio.");
        }
        this.codigo = codigo.trim();
    }

    public String getCurso() {
        return curso;
    }

    public final void setCurso(String curso) {
        if (curso == null || curso.trim().isEmpty()) {
            throw new IllegalArgumentException("El curso no puede estar vacio.");
        }
        this.curso = curso.trim();
    }

    public ArregloNotas getPracticas() {
        return practicas;
    }

    public void setPracticas(ArregloNotas practicas) {
        this.practicas = practicas;
    }

    /** Crea el arreglo donde se guardaran las notas de las practicas. */
    public void registrarCantidadPracticas(int cantidad) {
        if (cantidad < 2) {
            throw new IllegalArgumentException("Debe haber al menos 2 practicas para poder eliminar la mas
                baja.");
        }
        this.practicas = new ArregloNotas(cantidad);
    }

    @Override
    public String getRol() {
        return "Alumno";
    }

    // POLIMORFISMO DINAMICO
    @Override
    public void mostrarDatos() {
        System.out.println("=====");
        System.out.println(" FICHA DEL ALUMNO");
        System.out.println("=====");
        System.out.println("Alumno : " + getNombreCompleto());
        System.out.println("Codigo : " + codigo);
        System.out.println("Curso : " + curso);
        System.out.println("-----");
        practicas.escritura();
        System.out.println("-----");
        if (practicas.tieneDatos() && practicas.getTope() >= 2) {
            System.out.println("Nota mas baja eliminada : " + practicas.hallarMenor());
            System.out.println("PROMEDIO REDONDEADO : " + practicas.promedioRedondeado());
            System.out.println("Condicion : "
                + (practicas.estaAprobado() ? "APROBADO" : "DESAPROBADO"));
        } else {
            System.out.println("Aun no se puede calcular el promedio.");
        }
        System.out.println("=====");
    }
}
```

A.8 Docente.java

Hereda de Persona; demuestra el polimorfismo.

```
package Arreglo;

/**
 * HERENCIA: Docente tambien es una Persona, pero se muestra de otra forma.
 * Sirve para demostrar el POLIMORFISMO con el arreglo Persona[].
 */
public class Docente extends Persona {

    private String especialidad;

    public Docente(String nombres, String apellidos, String especialidad) {
        super(nombres, apellidos);
        setEspecialidad(especialidad);
    }

    public String getEspecialidad() {
        return especialidad;
    }

    public final void setEspecialidad(String especialidad) {
        if (especialidad == null || especialidad.trim().isEmpty()) {
            throw new IllegalArgumentException("La especialidad no puede estar vacia.");
        }
        this.especialidad = especialidad.trim();
    }

    @Override
    public String getRol() {
        return "Docente";
    }

    // POLIMORFISMO DINAMICO
    @Override
    public void mostrarDatos() {
        System.out.println("Docente del curso : " + getNombreCompleto()
            + " | Especialidad: " + especialidad);
    }
}
```

A.9 AplicaNotas.java

Clase principal con el método main y el menú.

```
package Arreglo;

import java.util.Locale;

/**
 * CLASE PRINCIPAL (main).
 *
 * ACTIVIDAD: ingresar y almacenar las notas de todas las practicas de un
 * alumno, determinar el promedio eliminando la practica con la nota mas baja,
 * mostrar el promedio redondeado y validar el ingreso de datos.
 */
public class AplicaNotas {

    // Referencia de tipo Persona apuntando a un Alumno (POLIMORFISMO)
    private static Alumno alumno = null;

    public static void main(String[] args) {
        System.out.println("*****");
        System.out.println("    SISTEMA DE NOTAS DE PRACTICAS - JAVA POO    ");
        System.out.println("*****");
        menu();
    }

    static void menu() {
        int opcion;

        do {
            System.out.println("            MENU DE OPCIONES");
            System.out.println("=====");
            System.out.println("1.- Registrar datos del alumno");
            System.out.println("2.- Registrar cantidad de practicas");
            System.out.println("3.- Ingresar notas de las practicas");
            System.out.println("=====");
            System.out.println("4.- Mostrar notas registradas");
            System.out.println("5.- Calcular promedio (sin la nota mas baja)");
            System.out.println("6.- Ficha completa del alumno");
        } while (opcion < 0 || opcion > 6);
    }
}
```

```

        System.out.println("=====");
        System.out.println("7.- Demostracion de P00 (herencia y polimorfismo)");
        System.out.println("0.- Salir");
        System.out.println("=====");

        opcion = ArregloBase.leerEnteroEnRango("Ingrese una alternativa (0-7) : ", 0, 7);
        System.out.println("");

        switch (opcion) {
            case 1 ->
                registrarAlumno();
            case 2 ->
                registrarPracticas();
            case 3 -> {
                if (hayAlumno()) {
                    alumno.getPracticas().ingresarNotas();
                }
            }
            case 4 -> {
                if (hayAlumno()) {
                    alumno.getPracticas().escritura();
                }
            }
            case 5 -> {
                if (hayAlumno()) {
                    alumno.getPracticas().mostrarPromedioPracticas();
                }
            }
            case 6 -> {
                if (hayAlumno()) {
                    alumno.mostrarDatos();
                }
            }
            case 7 ->
                demostracionP00();
            case 0 ->
                System.out.println("Saliendo del sistema ....");
        }
        System.out.println("");

    } while (opcion != 0);
}

/** Verifica que exista un alumno antes de operar. */
static boolean hayAlumno() {
    if (alumno == null) {
        System.out.println(">> Primero debe registrar los datos del alumno (opcion 1).");
        return false;
    }
    return true;
}

/** VALIDACION: ningun dato puede quedar vacio. */
static void registrarAlumno() {
    String nombres = ArregloBase.leerTexto("Nombres del alumno : ");
    String apellidos = ArregloBase.leerTexto("Apellidos del alumno : ");
    String codigo = ArregloBase.leerTexto("Codigo del alumno : ");
    String curso = ArregloBase.leerTexto("Curso : ");

    alumno = new Alumno(nombres, apellidos, codigo, curso);
    System.out.println(">> Alumno registrado: " + alumno.getNombreCompleto());
}

/** VALIDACION: minimo 2 practicas (se elimina una) y maximo 20. */
static void registrarPracticas() {
    if (!hayAlumno()) {
        return;
    }
    int cantidad = ArregloBase.leerEnteroEnRango(
        "Cantidad de practicas del curso (2-20) : ", 2, 20);
    alumno.registrarCantidadPracticas(cantidad);
    System.out.println(">> Se creo el arreglo para " + cantidad + " practicas.");
}

/**
 * Demostracion de los 4 pilares de la P00 en ejecucion.
 */
static void demostracionP00() {
    System.out.println("=====");
    System.out.println(" DEMOSTRACION DE PROGRAMACION ORIENTADA A OBJETOS");
    System.out.println("=====");

    // 1) POLIMORFISMO con la jerarquia Persona

```

```
System.out.println("-- 1. Polimorfismo con Persona[] --");
Docente docente = new Docente("Carlos", "Mendoza Rojas", "Algoritmica");
Persona[] personas = (alumno == null)
    ? new Persona[]{docente}
    : new Persona[]{alumno, docente};

for (Persona p : personas) {
    System.out.println("-> " + p);        // usa toString() sobrescrito
    p.mostrarDatos();                    // cada clase responde distinto
}

// 2) POLIMORFISMO con la jerarquia de arreglos
System.out.println("-- 2. Polimorfismo con ArregloBase[] --");
ArregloBase[] estructuras = (alumno == null)
    ? new ArregloBase[]{new Arreglos(), new ArregloEstadistico()}
    : new ArregloBase[]{new Arreglos(alumno.getPracticas()),
        new ArregloEstadistico(), alumno.getPracticas()};

for (ArregloBase a : estructuras) {
    a.mostrarInformacion();              // metodo abstracto implementado
}

// 3) POLIMORFISMO ESTATICO (sobrecarga de metodos)
System.out.println("-- 3. Sobrecarga de metodos --");
if (alumno != null && alumno.getPracticas().tieneDatos()
    && alumno.getPracticas().getTope() >= 2) {
    ArregloNotas n = alumno.getPracticas();
    System.out.println("promedioRedondeado() -> " + n.promedioRedondeado());
    System.out.println("promedioRedondeado(2) -> "
        + String.format(Locale.US, "%.2f", n.promedioRedondeado(2)));
} else {
    System.out.println("(Registre las notas para ver la sobrecarga en accion)");
}

// 4) ENCAPSULAMIENTO: el setter protege el objeto
System.out.println("-- 4. Encapsulamiento (el setter rechaza datos invalidos) --");
try {
    ArregloNotas prueba = new ArregloNotas(3);
    prueba.setTope(99);                  // 99 > capacidad 3
} catch (IllegalArgumentException e) {
    System.out.println("Excepcion controlada: " + e.getMessage());
}
System.out.println("=====");
}
}
```